Homework 3: indicizzazione e ricerca di file di testo con Lucene

Carmine Adduce
Matricola 589659
E-mail car.adduce@stud.uniroma3.it
Corso di Ingegneria dei Dati

Novembre 2026

Sommario

Questa relazione descrive un programma Java che indicizza con Apache Lucene i file `.txt` di una cartella, usando due campi (nome del file e contenuto) con un analyzer dedicato ciascuno, e che permette di interrogare l'indice da console con una sintassi semplice. Il sistema è stato sviluppato e provato con 21 query su un corpus bilingue (italiano e inglese) di 202 file, poi misurato su un corpus più grande di 2 810 file (circa 34 MB) costruito con uno script riproducibile da Wikipedia: l'indicizzazione richiede circa 2,3 secondi e il tempo cresce quasi linearmente con il numero di file. La qualità è valutata con precision e recall rispetto a una verità lessicale indipendente da Lucene (e controllata con `grep`).

Repository: `https://github.com/adducec03/lucene-file-indexer`

1 Obiettivo e tecnologie

La consegna chiedeva di:

1. indicizzare i file `.txt` di una cartella su due campi, nome e contenuto, scegliendo per ciascuno un analyzer appropriato;
2. leggere da console una query che inizia con la parola chiave del campo (`nome` o `contenuto`), seguita da termini eventualmente tra virgolette per esprimere una *phrase query*, e stamparne i risultati;
3. provare il sistema con una decina di query diverse.

Il progetto usa Java 21, Maven (con il wrapper `./mvnw`, che non richiede installazioni) e **Lucene 10.3.1**. Le dipendenze sono `lucene-core`, `lucene-analysis-common` e `lucene-queryparser`.
Oltre alla consegna, il sistema è stato provato anche su un corpus molto più grande di quello usato per lo sviluppo, per misurare come crescono i tempi di indicizzazione e per valutare la qualità dei risultati con metriche (precision e recall), non solo contando i file trovati.

2 Architettura

Il codice è nel package `it.homework.lucene` ed è diviso in cinque classi, ciascuna con un solo compito (tabella 1), più una sesta classe, `Evaluator`, che serve solo per la valutazione (sezione 8). I due flussi principali, indicizzazione e ricerca, condividono lo stesso analyzer: è essenziale, perché i termini della query devono essere trattati esattamente come quelli dell'indice (figura 1).

Classe	Responsabilità
`Main`	comandi `index` e `search`, ciclo interattivo su console
`Analyzers`	nomi dei campi e analyzer per campo (`PerFieldAnalyzerWrapper`)
`FileIndexer`	visita la cartella, crea un `Document` per file, misura i tempi
`QueryBuilder`	traduce la riga digitata in una `Query` di Lucene
`FileSearcher`	apre l'indice in lettura, esegue la query e stampa i risultati
`Evaluator`	(solo valutazione) calcola precision e recall confrontando i risultati con una verità lessicale

Tabella 1: Le classi del progetto.

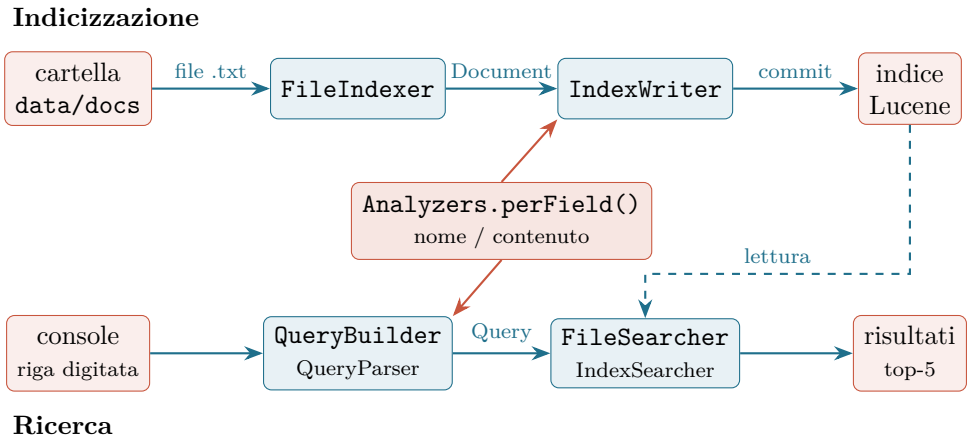


Figura 1: I due flussi del programma. Lo stesso analyzer per campo (in rosso) viene usato sia per scrivere l'indice sia per analizzare la query.

3 Dataset

Sono stati usati due corpora. Il **corpus piccolo** (202 file) è servito per lo sviluppo e per le 21 query di test; il **corpus grande** (2 810 file) è servito per misurare i tempi su molti documenti e per valutare precision e recall. Entrambi sono testi di Wikipedia in italiano e in inglese, scaricati con le API pubbliche di Wikipedia; i testi sono distribuiti con licenza CC BY-SA. Non sono un benchmark pubblico, ma servono a provare l'indicizzazione su testo reale.

3.1 Corpus piccolo (sviluppo e test)

Il corpus piccolo contiene **200 articoli di Wikipedia scelti a caso**, 100 in italiano e 100 in inglese, scaricati tramite le API di Wikipedia. Sono stati scartati gli articoli troppo corti, sotto i 5000 caratteri di testo. A questi si aggiungono due piccoli file scritti a mano (`example1.txt`, `example2.txt`). I testi di Wikipedia sono distribuiti con licenza CC BY-SA.

Ogni file contiene il titolo dell'articolo nella prima riga e poi il testo in chiaro. Il nome del file è `<lingua>_<Titolo_con_underscore>.txt` (ad esempio `en_History_of_martial_arts.txt`), così il campo `nome` contiene parole significative da cercare.

3.2 Corpus grande (tempi e valutazione)

Il corpus grande è stato costruito con lo script `tools/build_corpus.py` (Python, solo libreria standard), incluso nel repository, in modo che chiunque possa rigenerarlo. Il procedimento è il seguente:

	File	Parole	Dimensione
Italiano	100	215 394	1,44 MB
Inglese	100	209 421	1,32 MB
Esempi	2	23	$<$ 1 KB
Totale	**202**	**424 838**	**2,76 MB**

Tabella 2: Composizione del corpus. La dimensione mediana di un file è di circa 10 KB, la massima di 103 KB.

1. **Articoli per argomento.** Per ciascuna lingua sono state scelte 14 categorie di Wikipedia (terremoti, battaglie della seconda guerra mondiale, guerre napoleoniche, squadre di calcio italiane, teoremi, pittori italiani, composti chimici, vulcani, fiumi italiani, imperatori romani, malattie infettive, opere liriche, aerei militari e linguaggi di programmazione). Dalle categorie, scendendo nelle sottocategorie fino a due livelli, si prendono fino a 80 articoli per argomento e per lingua. Così il corpus contiene gruppi di documenti sullo stesso tema, con parole che ricorrono in molti file e parole rare.

2. **Articoli casuali.** Altri 400 articoli casuali per lingua (369 italiani e 346 inglesi effettivamente salvati) fanno da rumore di fondo e sono su qualunque argomento.

3. **Filtri e formato.** Sono scartati gli articoli sotto i 3 000 caratteri. Ogni file contiene il titolo e il testo in chiaro e si chiama `<lingua>_<Titolo_con_underscore>.txt`, come nel corpus piccolo.

4. **Manifest.** Il file `data/corpus-manifest.csv` elenca per ogni articolo lingua, titolo, nome del file, argomento di provenienza, lunghezza e categorie di Wikipedia. Descrive com'è fatto il corpus.

	File	Parole	Dimensione
Italiano	1 401	2 419 562	16,2 MB
Inglese	1 409	2 868 253	18,2 MB
Totale	**2 810**	**5 287 815**	**34,4 MB**

Tabella 3: Composizione del corpus grande. La dimensione mediana di un file è di circa 6,9 KB, la massima di 145 KB.

Argomento	it	en	Argomento	it	en
Terremoti	74	80	Fiumi	54	27
Battaglie 2ª g. m.	80	80	Imperatori romani	80	80
Guerre napoleoniche	80	80	Malattie infettive	80	80
Squadre di calcio	80	80	Opere liriche	80	80
Teoremi	80	80	Aerei militari	80	80
Pittori	80	80	Linguaggi di prog.	80	80
Composti chimici	41	80	Articoli casuali	369	346
Vulcani	63	76			

Tabella 4: Numero di articoli per argomento e lingua. Alcune categorie italiane (chimica, fiumi, vulcani, terremoti) contengono meno di 80 articoli abbastanza lunghi.

4 Analyzer scelti

I due campi hanno caratteristiche molto diverse, quindi ricevono analyzer diversi, combinati con un `PerFieldAnalyzerWrapper`. L'analyzer di default del wrapper (`StandardAnalyzer`) non viene mai usato, perché i campi sono solo due. Gli analyzer sono stati scelti in due passi: una prima versione, che il confronto con `grep` ha mostrato inadeguata, e la versione definitiva.

4.1 Prima versione e problemi riscontrati

La prima versione era la scelta più immediata:

- `contenuto`: `StandardAnalyzer` con l'*unione* delle stopword italiane e inglesi di Lucene;
- `nome`: `StandardTokenizer`, `LowerCaseFilter` e `WordDelimiterGraphFilter`.

Sembrava ragionevole e le prime prove funzionavano. Confrontando però il numero di file trovati con quello di `grep -ilw` sui file originali, sono emersi **molti file che il sistema non trovava**:

Termine cercato	`grep` (reale)	Prima versione	Versione definitiva
`uomo`	19	10	19
`italia`	38	28	38
`impero`	10	5	10
`anno`	55	43	53
`king`	21	18	21
`citta` (senza accento)	34	0	34
`era`	95	nessun termine utile	95

Tabella 5: File trovati per alcuni termini, con la prima versione degli analyzer e con quella definitiva, a confronto con il numero reale ottenuto con `grep`.

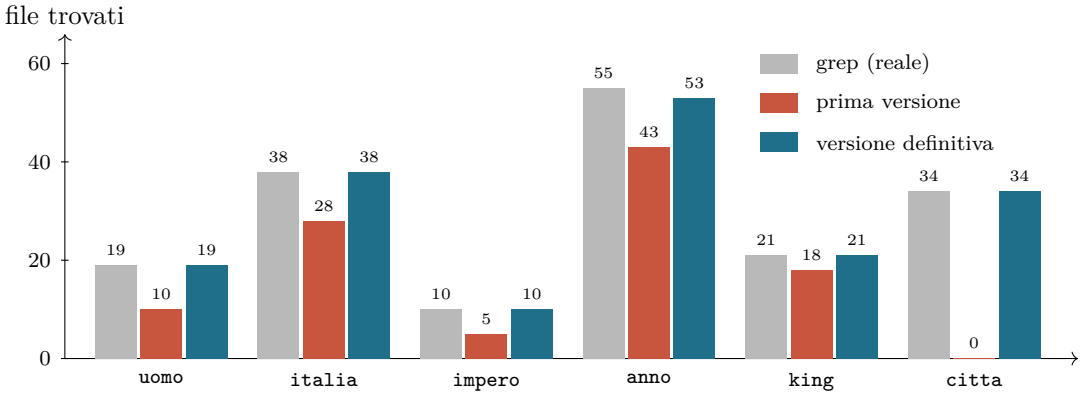


Figura 2: File trovati per termine: nella prima versione una parte dei file, a volte tutti, andava persa.

Le cause sono tre, tutte verificate con prove dirette sugli analyzer.

1. Gli apostrofi non separano le parole. Lo `StandardTokenizer` segue le regole Unicode e non spezza su un apostrofo tra due lettere: `dell'uomo` resta *un solo token*, e così `l'Italia` e `nell'anno`. Nel testo italiano di Wikipedia queste forme sono continue (`dell'anno` compare 28 volte, `l'anno` 26), quindi cercare `uomo` non trovava i file in cui compare solo come `dell'uomo` o `l'uomo`: 10 file invece di 19. Lo stesso avviene per il possessivo inglese (`king's` resta un token a sé), con 18 file invece di 21.

2. Gli accenti non sono normalizzati. `città` e `citta` sono termini diversi: cercando senza accento, come si fa spesso, la prima versione trovava **0 file** contro i 34 reali.

3. Le stopword delle due lingue si scontrano. L'unione delle liste elimina da *tutti* i testi le parole vuote di entrambe le lingue. Ma 27 stopword italiane compaiono anche nel dizionario inglese del sistema, e alcune sono parole inglesi comuni, per esempio *era*, *dove* e *all*. Con la prima versione la query `contenuto era` rispondeva "nessun termine utile", pur esistendo 95 file che contengono la parola.

Campo `nome`. Il `WordDelimiterGraphFilter` separava anche lettere e numeri: `example1` diventava `example` e `1`, e con l'operatore OR la query `nome example1` restituiva 4 file, tre dei quali trovati solo per una parte del termine: `example2` per `example`, `en_2025_GN1` e `it_De_Havilland_Canada_DHC_1_Chipmunk` per il numero `1`.

Lezione appresa: un analyzer non si valuta guardando i token di due frasi di prova, ma confrontando il numero di risultati con una verità indipendente (qui `grep`) su termini reali del corpus.

4.2 Versione definitiva: campo `nome`

Scelta: un `CustomAnalyzer` con un `CharFilter` che sostituisce `_` con uno spazio, poi `StandardTokenizer`, `LowerCaseFilter` e `ASCIIFoldingFilter`; senza stopword e senza stemming.

Motivazione:

- i nomi dei file usano spesso `_`, `-` e `.` come separatori. Lo `StandardTokenizer` spezza su trattini e punti ma *non* su `_`, che lo standard Unicode considera parte della parola: il `CharFilter` lo trasforma in uno spazio *prima* del tokenizer, e il problema è risolto;
- si è tolto il `WordDelimiterGraphFilter`: lettere e numeri non vengono più separati, quindi `example1` resta un unico termine;
- minuscolo e normalizzazione degli accenti rendono la ricerca indipendente da maiuscole e accenti;
- nei nomi anche le parole brevi (*di*, *the*) possono essere significative, e lo stemming altererebbe nomi propri e sigle, quindi non si usano né stopword né stemming;
- l'estensione `.txt` viene tolta prima dell'indicizzazione: altrimenti `txt` sarebbe un termine presente in ogni documento.

4.3 Versione definitiva: campo `contenuto`

Scelta: un analyzer composto da `StandardTokenizer`, `ElisionFilter` (articoli italiani), `EnglishPossessiveFilter`, `LowerCaseFilter` e `ASCIIFoldingFilter`; **senza stopword e senza stemming**.

Motivazione: il corpus è bilingue.

- i filtri aggiunti risolvono i primi due problemi della sezione 4.1: l'elisione toglie l'articolo davanti all'apostrofo (`dell'uomo` → `uomo`), il filtro del possessivo toglie `'s` (`king's` → `king`) e la normalizzazione degli accenti rende uguali `città` e `citta`;
- **nessuno stemming**: un `ItalianAnalyzer` o un `EnglishAnalyzer` applicherebbero lo stemming di una lingua anche ai testi dell'altra, storpiandone le parole;

- **nessuna lista di stopword**: senza riconoscere la lingua di ogni testo, le liste delle due lingue si scontrano (terzo problema). Il ranking BM25 abbassa già da solo il peso dei termini molto frequenti, quindi la perdita di precisione è piccola, come mostrano i test (query 12 e 18). Le alternative valutate, con le misure, sono nella sezione 4.4.

```
// nome
CustomAnalyzer.builder()
    .addCharFilter(PatternReplaceCharFilterFactory.class,
                   "pattern", "_", "replacement", " ")
    .withTokenizer(StandardTokenizerFactory.class)
    .addTokenFilter(LowerCaseFilterFactory.class)
    .addTokenFilter(ASCIIFoldingFilterFactory.class)
    .build();

// contenuto
Tokenizer source = new StandardTokenizer();
TokenStream chain = new ElisionFilter(source, ITALIAN_ARTICLES);
chain = new EnglishPossessiveFilter(chain);
chain = new LowerCaseFilter(chain);
chain = new ASCIIFoldingFilter(chain);
```

Componente	Cosa fa	Esempio
`PatternReplaceCharFilter`	sostituisce `_` con uno spazio prima del tokenizer	`Finale_2024` → `Finale 2024`
`StandardTokenizer`	divide sui confini di parola (Unicode), scarta la punteggiatura	`e-mail,` → `e`, `mail`
`ElisionFilter`	toglie l'articolo davanti all'apostrofo, per gli articoli italiani dell'elenco	`dell'uomo` → `uomo`
`EnglishPossessiveFilter`	toglie il possessivo inglese	`dog's` → `dog`
`LowerCaseFilter`	porta in minuscolo	`Roma` → `roma`
`ASCIIFoldingFilter`	toglie gli accenti	`città` → `citta`

Tabella 6: I componenti degli analyzer definitivi.

4.4 Alternative valutate

Prima di eliminare le stopword dalla versione definitiva sono state confrontate quattro varianti dello stesso analyzer di `contenuto`, cambiando solo l'elenco delle parole eliminate (tabella 7). Per ciascuna è stato costruito l'indice dei 202 file e ne sono stati misurati la dimensione su disco e il tempo di indicizzazione.

Variante	Parole eliminate	Indice	Tempo
Nessuna stopword (**scelta finale**)	0	1299 KB	148 ms
Solo articoli (*il, lo, la, gli, le, un, uno, una, the, a, an*)	11	1261 KB	132 ms
Lista selettiva (tutte tranne 8 parole ambigue)	302	1133 KB	124 ms
Lista completa (italiano + inglese, prima versione)	310	1128 KB	123 ms

Tabella 7: Effetto dell'elenco di stopword su dimensione dell'indice e tempo di indicizzazione (202 file).

Lista completa. Riduce l'indice del 13% ma, come visto nella sezione 4.1, elimina da tutti i testi parole che nell'altra lingua sono significative.

Solo articoli. Elimina solo le parole più innocue, ma il vantaggio è minimo (3%): lo spazio è occupato soprattutto da preposizioni e verbi ausiliari (*di*, *che*, *avere*, *essere*).

Lista selettiva. Per escludere le parole ambigue si è misurata, per ogni stopword, la frequenza (ogni 10 000 parole) nei testi italiani e in quelli inglesi. Solo 7 parole risultano frequenti nella lingua sbagliata (*all*, *i*, *era*, *ed*, *c*, *io*, *mi*), alle quali si è aggiunta *dove*. Con queste esclusioni le prestazioni sono praticamente quelle della lista completa (1133 contro 1128 KB). Il metodo ha però un limite: restano eliminate parole che nei testi inglesi esistono davvero, anche se meno frequenti che in italiano (tabella 8). Ad esempio `non` compare in 29 file inglesi, in gran parte perché il tokenizer spezza i composti come *non-profit*; chi cerca `non` in un testo inglese non troverebbe il file. Inoltre l'elenco dipende dal corpus e andrebbe rifatto per testi diversi.

Stopword italiana	File inglesi	File italiani
`non`	29	86
`per`	22	100
`come`	16	93

Tabella 8: Stopword italiane che la lista selettiva elimina (non risultano ambigue nella misura di frequenza) ma che compaiono comunque in molti file inglesi.

Riconoscimento della lingua. L'approccio corretto sarebbe applicare l'analyzer (e la lista di stopword) della lingua di ciascun documento. La lingua si riconosce bene anche senza librerie: contando nel testo alcune parole funzionali (*di, e, il, la, che, per, ...* contro *the, of, and, to, in, ...*) e scegliendo la lingua con la percentuale più alta, sui 200 articoli di Wikipedia la lingua è stata indovinata in **200 casi su 200**, con un margine minimo di 6 punti percentuali tra le due lingue. Usarla in Lucene è però più complesso: l'analyzer è associato al campo e non al documento, quindi servirebbero un campo per lingua (`contenuto_it`, `contenuto_en`) oppure un `TokenStream` costruito per ogni documento, e le query dovrebbero essere analizzate con entrambi gli analyzer, perché la lingua di una query di una o due parole non è affidabile.

Conclusione. Si è scelto di **non usare stopword**. Il vantaggio della lista completa è piccolo (13% di spazio, pochi millisecondi su 202 file, un guadagno che diventa rilevante solo su grandi collezioni), mentre il costo è una ricerca scorretta; la lista selettiva ridurrebbe il costo ma non lo azzera ed è legata al corpus; il riconoscimento della lingua è la soluzione più corretta, ma richiede molto codice in più per un risultato che con l'analyzer attuale è già verificato (i totali coincidono con `grep`). Il ranking BM25 dà già un peso molto basso ai termini molto frequenti, quindi la precisione non ne risente in modo sensibile. Il riconoscimento della lingua resta come possibile estensione (sezione 9).

4.5 Verifica

La figura 3 riporta i token realmente prodotti dagli analyzer definitivi (output del programma di prova).

4.6 Limiti noti

- **Nessuno stemming:** cercare *libro* non trova *libri*. È il prezzo della scelta multilingua.

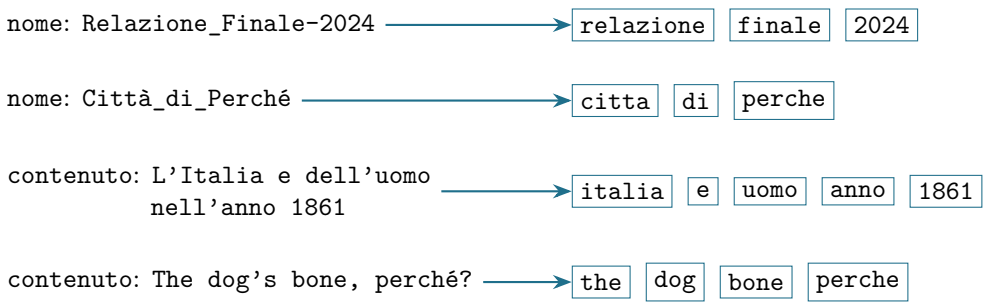


Figura 3: Token prodotti dagli analyzer definitivi. Il carattere `_` separa le parole, gli articoli elisi e il possessivo vengono tolti, accenti e maiuscole normalizzati; le parole comuni (*e*, *the*) restano.

- **Nessuna stopword:** una query fatta solo di parole comuni, come `contenuto il the`, trova quasi tutti i file (199 su 202, e 2 803 su 2 810 nel corpus grande); il ranking li mette comunque in ordine di rilevanza, ma l'utente non viene avvisato.
- **Elenco degli articoli incompleto:** `quell'anno` (7 occorrenze) non viene trattato perché `quell` non è nell'elenco, e per questo `anno` trova 53 file invece di 55 (nel corpus grande la valutazione trova casi analoghi: *quest'uomo*, *pover'uomo*, *quand'era*, *com'era*).
- **Indirizzi web nel testo:** lo `StandardTokenizer` non spezza una parola su un punto tra due lettere, per cui in un testo con `volcano.und` il termine *volcano* non viene trovato.
- **Numeri nei nomi:** `example1` è un unico termine, quindi la query `nome example` non trova più il file `example1`. È il rovescio della scelta fatta per evitare i risultati spuri.

5 Indicizzazione

Si crea un `Document` Lucene per ogni file, con due campi (tabella 9). La cartella viene visitata ricorsivamente e vengono presi i file con estensione `.txt`, senza distinguere maiuscole e minuscole. L'indice è ricreato da zero a ogni esecuzione (`OpenMode.CREATE`), così rieseguire il programma non duplica i documenti. Un file illeggibile (ad esempio non codificato in UTF-8) viene saltato con un avviso e contato a parte, senza interrompere il lavoro. Alla fine viene fatto il `commit()` e vengono stampati il numero di file e il tempo impiegato.

Campo	Tipo	Stored	Perché
`nome`	`TextField`	sì	serve per stampare il nome nei risultati
`contenuto`	`TextField`	no	serve solo per cercare: non salvarlo riduce lo spazio occupato dall'indice

Tabella 9: I campi del documento Lucene.

5.1 Risultati sul corpus piccolo

Sul corpus piccolo, su cinque esecuzioni consecutive con gli analyzer definitivi sono stati indicizzati **202 file**, con **0 file saltati**, in circa **0,6 secondi** (figura 4). L'indice occupa circa 1,3 MB, contro i 2,8 MB di testo. La prima versione degli analyzer impiegava circa 0,5 secondi (tre esecuzioni: 565, 466 e 477 ms) e produceva un indice di circa 1,2 MB: i filtri aggiunti e l'assenza di stopword costano poco, dello stesso ordine di grandezza.

Ambiente di misura: MacBook Pro (2020) con chip Apple M1 a 8 core (4 prestazioni e 4 efficienza), 8 GB di RAM, macOS 26.6, OpenJDK 21.0.10.

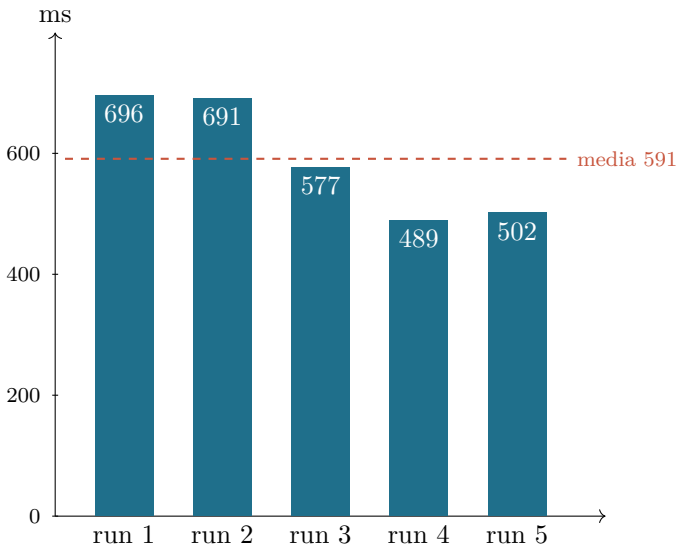


Figura 4: Tempo totale di indicizzazione dei 202 file in cinque esecuzioni.

Il tempo misurato comprende la lettura dei file e l'analisi del testo ed è preso in una JVM appena avviata, senza riscaldamento: le prime esecuzioni sono le più lente. Per questo conviene considerare l'ordine di grandezza (circa 0,5–0,7 s) più del valore singolo. Il costo per file (2–3 ms) è però sovrastimato, perché su pochi file pesa molto il tempo fisso di avvio di JVM e Lucene: lo mostra la misura sul corpus grande.

5.2 Tempi al crescere del corpus

Per capire come crescono i tempi, lo script `tools/benchmark_indexing.py` costruisce sottoinsiemi casuali (con seme fisso, quindi sempre uguali) del corpus grande da 200, 500, 1 000, 2 000 e 2 810 file e lancia l'indicizzatore cinque volte per ciascuno, ogni volta in una nuova JVM. Nella tabella 10 si riporta la **mediana**: una delle cinque esecuzioni sul corpus completo è durata 3 289 ms contro circa 2 150 ms delle altre (probabilmente un disturbo del sistema) e la mediana non ne risente.

File	Testo (MB)	Indice (MB)	Mediana (ms)	ms/file
200	2,1	1,05	461	2,31
500	5,7	2,52	705	1,41
1 000	12,0	4,86	1 013	1,01
2 000	23,9	9,63	1 664	0,83
2 810	34,4	13,73	2 326	0,83

Tabella 10: Tempo di indicizzazione al crescere del corpus (mediana di cinque esecuzioni).

La crescita è quasi lineare. Il tempo è circa un costo fisso di 300 ms (avvio di JVM e Lucene) più circa **0,7 ms per file** (stima dai punti a 200 e a 2 810 file). Per questo il costo medio per file scende con la dimensione del corpus e oltre i 2 000 file si stabilizza a circa 0,83 ms (colonna *ms/file*), cioè circa 15 MB di testo al secondo. L'indice occupa circa il 40% del testo, perché il contenuto non è memorizzato. Indicizzare tutti i 2 810 file richiede quindi circa 2,3 secondi.

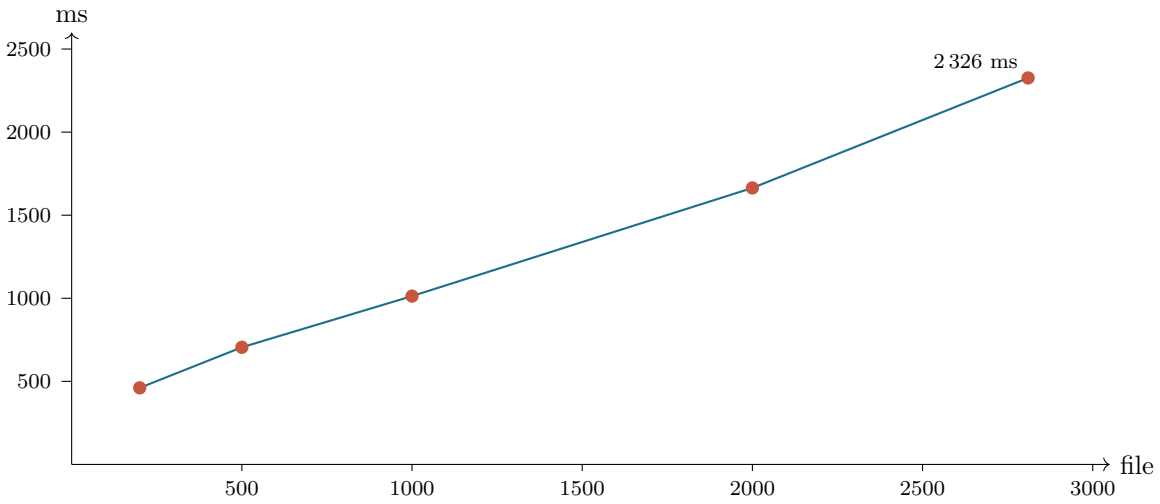


Figura 5: Tempo di indicizzazione in funzione del numero di file (mediana).

6 Ricerca

6.1 Sintassi della query

Ogni riga digitata ha la forma `<campo> <termini>`, dove `<campo>` è `nome` o `contenuto`. Il testo tra virgolette è una frase. Ad esempio:

```
nome knitting
contenuto guerra mondiale
contenuto "world war"
```

6.2 Costruzione della query

La parola chiave iniziale viene separata a mano (`split(" ", 2)`), mentre il resto della riga è affidato al `QueryParser` di Lucene, configurato con il campo scelto come *campo di default* e con lo stesso analyzer usato in indicizzazione. In questo modo le frasi tra virgolette e l'analisi dei termini sono gestite da Lucene, con meno codice e meno casi limite rispetto a un parser scritto a mano.

Più termini senza virgolette sono in OR. È il comportamento predefinito del `QueryParser`: i termini senza modificatori sono *opzionali* (`BooleanClause.Occur.SHOULD`). I documenti che contengono più termini ottengono comunque un punteggio maggiore. Il ranking è quello predefinito di Lucene (BM25).

Riga digitata	Query Lucene generata
`nome roma`	`nome:roma`
`contenuto data scientist`	`contenuto:data contenuto:scientist`
`contenuto "data scientist"`	`contenuto:"data scientist"`
`contenuto Data`	`contenuto:data`
`nome Relazione_Finale`	`nome:relazione nome:finale`

Tabella 11: Traduzione delle righe digitate (output reale di `query.toString()`).

6.3 Esecuzione e risultati

Il `FileSearcher` apre l'indice con `DirectoryReader` e `IndexSearcher` e viene chiuso automaticamente a fine sessione (`try-with-resources`). Per ogni query stampa il numero totale di file trovati e i primi **5** (posizione, nome e punteggio). Il nome viene letto dal campo memorizzato; il contenuto non si può mostrare perché non è memorizzato.

Gli errori di input sono gestutu: campo sconosciuto, testo mancante e sintassi non valida (ad esempio virgolette non chiuse) producono un messaggio e il ciclo prosegue. Se tutti i termini vengono eliminati dall'analyzer (ad esempio una query fatta solo di simboli, come `c++`) la query è vuota e il programma lo segnala invece di restituire zero risultati senza spiegazione.

7 Test del sistema

Le 21 query di test sono eseguite sul **corpus piccolo** (202 file). Sono raccolte nel file `queries.txt` ed eseguite in modo automatico dallo script `run-queries.sh`, che salva l'output completo in `results/query-results.md`. I test sono quindi riproducibili. Per le query sul contenuto con un termine o una frase (query 5–11 e 14–18) il numero di risultati è stato confrontato con quello di `grep -il` sui file originali: in tutti i casi coincide. Le query 14–18 sono state aggiunte per verificare i problemi della sezione 4.1.

Perché il corpus piccolo. Questi test sono *funzionali*: mostrano il comportamento del programma (output, top-5, frasi, casi di errore) e non dipendono dalla dimensione del corpus. Si usa il corpus piccolo perché con 202 file il numero di risultati si controlla facilmente a mano con `grep`, e perché i problemi della sezione 4.1 sono stati individuati e corretti su quei file, quindi i numeri si possono confrontare direttamente. La correttezza su larga scala, con 2 810 file e un riferimento indipendente da Lucene, è valutata nella sezione 8.

#	Query	Cosa verifica	Ris.	Primo risultato
1	`nome knitting`	un termine nel nome	1	`en_Knitting`
2	`nome history martial`	più termini nel nome (OR)	1	`en_History_of_martial...`
3	`nome "martial arts"`	frase nel nome	1	`en_History_of_martial...`
4	`nome list`	totale oltre il top-5	6	`en_List_of_formulae...`
5	`contenuto napoleone`	termine italiano	5	`it_Castello_di_Saint...`
6	`contenuto earthquake`	termine inglese	1	`en_2020_Petrinja...`
7	`contenuto "world war"`	frase nel contenuto	16	`en_Protectorate_of...`
8	`contenuto guerra mondiale`	più termini (OR)	44	`it_Polemiko_Nautiko`
9	`contenuto NAPOLEONE`	maiuscole ignorate	5	come la query 5
10	`contenuto "data scientist"`	frase in un solo file	1	`example1`
11	`contenuto xyzqwerty`	nessun risultato	0	–
12	`contenuto il the`	solo parole comuni	199	`it_Emily_Blunt`
13	`nome example1`	numero nel nome	1	`example1`
14	`contenuto uomo`	elisione (*dell'uomo*)	19	`it_Belle_Gunness`
15	`contenuto italia`	elisione (*l'Italia*)	38	`it_Giuliano_Ferrara`
16	`contenuto citta`	accenti (*città*)	34	`it_Citta_di_Ebla`
17	`contenuto king`	possessivo (*king's*)	21	`it_Non_siamo_piu...`
18	`contenuto era`	parola italiana e inglese	95	`it_Detlev_Peukert`
19	`titolo knitting`	campo sconosciuto	–	errore gestito
20	`contenuto`	testo mancante	–	errore gestito
21	`contenuto "frase aperta`	virgolette non chiuse	–	errore gestito

Tabella 12: Le 21 query di test (risultati reali, con gli analyzer definitivi).

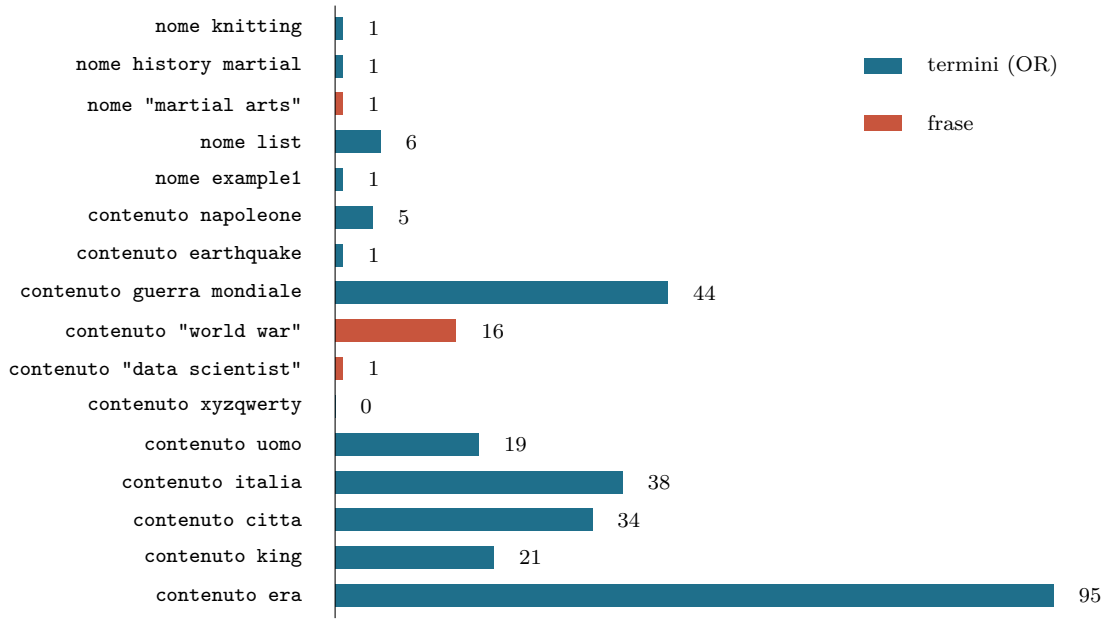


Figura 6: Numero di file trovati dalle query di test (in rosso le frasi tra virgolette). Sono escluse la query 9, che duplica la 5, la query 12 (199 file) e i casi di errore 19–21.

7.1 Osservazioni

- **Problemi risolti.** Le query 14–18 danno esattamente il numero di file di `grep` (19, 38, 34, 21, 95): elisioni, accenti, possessivi e parole come *era* non causano più file mancanti. La query 13 trova solo `example1`, senza i risultati spuri della prima versione.
- **OR contro frase.** `contenuto guerra mondiale` trova 44 file, cioè tutti quelli che contengono almeno una delle due parole, mentre la frase `"world war"` ne trova 16: la frase è molto più precisa. Il ranking BM25 porta comunque in cima i file che contengono entrambe le parole più volte.
- **Analyzer coerente tra indice e query.** Le query 5 e 9 danno esattamente gli stessi risultati e gli stessi punteggi: il minuscolo è applicato sia ai documenti sia alla query.
- **Parole comuni.** Senza stopword, la query 12 trova 199 file su 202: è il costo della scelta, mitigato dal fatto che BM25 dà poco peso ai termini molto frequenti.
- **Casi di errore.** Le query 19–21 non fanno terminare il programma: ognuna produce un messaggio comprensibile.

8 Valutazione con precision e recall

Contare i file trovati non dice se ne mancano. Per valutare la *recall* serve un riferimento indipendente da Lucene che dica quali file *dovrebbero* essere trovati. La valutazione è fatta sul corpus grande (2 810 file) dalla classe `Evaluator`.

8.1 Metodo

Verità lessicale. Per ogni query, un file è *rilevante* se contiene davvero la parola cercata (o, per una frase tra virgolette, le parole una dopo l'altra). L'insieme dei file rilevanti è calcolato senza usare Lucene: il programma legge ogni file, toglie gli accenti, porta il testo in minuscolo, lo divide su ogni carattere che non sia una lettera o una cifra e controlla se i termini della query compaiono,

come farebbe `grep -iw`. Più termini senza virgolette sono in OR, come nel `QueryParser`. Per il campo `nome` si fa lo stesso sul nome del file.

Metriche. Per ogni query, su *tutti* i risultati:

$$\text{precision} = \frac{\text{file trovati e rilevanti}}{\text{file trovati}}, \qquad \text{recall} = \frac{\text{file trovati e rilevanti}}{\text{file rilevanti}},$$
$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Se non esistono file rilevanti e non ne viene trovato nessuno la risposta è corretta (precision e recall pari a 1); se ne esistono ma non ne viene trovato nessuno la precision è posta a 0. Si riporta la media sulle query (media macro).

Cosa si confronta. Le stesse 26 query (20 sul contenuto e 6 sul nome), con gli stessi file, sono eseguite con due configurazioni: la *prima versione* degli analyzer (sezione 4.1) e la *versione definitiva*. Le query comprendono termini italiani e inglesi, frasi, parole con elisione o accento, parole comuni e casi sul nome.

Controllo dell'oracolo. Poiché la verità è scritta da noi, i suoi insiemi sono stati ricalcolati con altri due strumenti (`perl` per minuscolo e accenti, `grep -E` per la ricerca) con lo script `tools/crosscheck_oracle.py`: i file rilevanti coincidono **su tutte le 26 query, file per file**.

8.2 Risultati

		Prima versione			**Definitiva**		
Query	**Ril.**	**Trov.**	**P**	**R**	**Trov.**	**P**	**R**
`contenuto napoleone`	83	83	1,00	1,00	83	1,00	1,00
`contenuto napoleon`	100	83	1,00	0,83	100	1,00	1,00
`contenuto terremoto`	109	109	1,00	1,00	109	1,00	1,00
`contenuto earthquake`	140	140	1,00	1,00	140	1,00	1,00
`contenuto vulcano`	74	74	1,00	1,00	74	1,00	1,00
`contenuto volcano`	100	99	1,00	0,99	99	1,00	0,99
`contenuto "world war"`	349	349	1,00	1,00	349	1,00	1,00
`contenuto "seconda guerra mondiale"`	186	186	1,00	1,00	186	1,00	1,00
`contenuto "roman empire"`	119	119	1,00	1,00	119	1,00	1,00
`contenuto "machine learning"`	12	12	1,00	1,00	12	1,00	1,00
`contenuto impero`	209	84	1,00	0,40	209	1,00	1,00
`contenuto uomo`	211	133	1,00	0,63	209	1,00	0,99
`contenuto italia`	518	427	1,00	0,82	518	1,00	1,00
`contenuto citta`	535	2	1,00	0,00	535	1,00	1,00
`contenuto king`	273	257	1,00	0,94	273	1,00	1,00
`contenuto era`	1030	0	0,00	0,00	1027	1,00	1,00
`contenuto anno`	540	382	1,00	0,71	522	1,00	0,97
`contenuto NAPOLEONE`	83	83	1,00	1,00	83	1,00	1,00
`contenuto teorema theorem`	152	152	1,00	1,00	152	1,00	1,00
`contenuto il the`	2803	0	0,00	0,00	2803	1,00	1,00
`nome napoleone`	2	2	1,00	1,00	2	1,00	1,00
`nome roma`	7	7	1,00	1,00	7	1,00	1,00
`nome battle`	15	15	1,00	1,00	15	1,00	1,00
`nome "world war"`	10	10	1,00	1,00	10	1,00	1,00
`nome example1`	0	10	0,00	1,00	0	1,00	1,00
`nome vesuvio`	1	1	1,00	1,00	1	1,00	1,00
Media			0,88	0,82		1,00	1,00

Tabella 13: Precision (P) e recall (R) sul corpus grande, per ogni query. *Ril.* è il numero di file rilevanti per la verità lessicale, *Trov.* il numero di file trovati da Lucene. Nell'ultima riga la media macro sulle 26 query.

La media macro passa, dalla prima versione alla definitiva, da precision 0,88 e recall 0,82 (F_1 0,81) a precision 1,00 e recall 1,00 (F_1 1,00), con arrotondamento a due cifre. I casi più chiari sono:

- `contenuto citta`: la prima versione trova solo 2 dei 535 file (recall 0,00) perché non normalizza gli accenti, la definitiva li trova tutti;
- `contenuto era`: 0 su 1 030, perché *era* è una stopword italiana; la definitiva trova 1 027 file su 1 030;
- `contenuto impero`, `uomo`, `italia`: la prima versione perde i file in cui la parola segue un articolo con apostrofo (*dell'impero*, *l'uomo*, *l'Italia*), con recall rispettivamente 0,40, 0,63 e 0,82; `napoleon` (recall 0,83) perde 17 file su 100: 10 in cui compare solo come possessivo inglese (*Napoleon's*, per lo più in titoli di libri nelle bibliografie), 6 in cui compare solo con l'accento (*Napoléon*, in citazioni francesi) e 1 in cui compare solo in un indirizzo web (`napoleon.org`);
- `nome example1`: la prima versione restituisce 10 file non pertinenti, perché `WordDelimiterGraphFilter` separa *example1* in *example* e *1* e cerca il numero 1 in molti nomi; nel corpus non esiste nessun file con quel nome, quindi la risposta corretta è zero risultati, che la versione definitiva dà.

La precision della versione definitiva è 1,00 su tutte le query: ogni file trovato contiene davvero la parola.

8.3 Limiti della valutazione

La versione definitiva non è perfetta: perde da 1 a 18 file su alcune query. Aprendo i file corrispondenti si trovano due cause. La prima sono le **elisioni non presenti nell'elenco degli articoli**: *quest'uomo*, *pover'uomo*, *quand'era*, *com'era*, *quell'anno*; così `uomo` trova 209 file su 211, `era` 1 027 su 1 030 e `anno` 522 su 540. La seconda sono gli **indirizzi web** come `volcano.und`, che lo `StandardTokenizer` non spezza. Entrambe sono descritte tra i limiti.

Che cosa misura questa valutazione. La verità lessicale controlla che l'indicizzazione sia corretta, cioè che non vengano persi file che contengono davvero la parola; non misura se i risultati siano *utili* per chi cerca (un articolo che nomina Napoleone una volta sola è comunque rilevante). Inoltre la verità usa regole simili a quelle dell'analyzer (minuscolo, niente accenti, separazione sui simboli), quindi l'accordo quasi totale dimostra che l'analyzer applica correttamente queste regole e non che le regole siano le uniche possibili. Il confronto tra le due versioni resta valido perché entrambe sono misurate con lo stesso riferimento. Per questo motivo non si riporta la precision@5: con questa verità ogni file trovato è rilevante, e valutare l'ordine dei risultati richiederebbe una verità tematica (per esempio le categorie di Wikipedia, già presenti nel manifest), non realizzata.

9 Limiti e possibili miglioramenti

- **Stemming e stopword per lingua:** riconoscere la lingua di ogni file e usare l'analyzer della lingua giusta migliorerebbe il richiamo (*libro*/*libri*) e permetterebbe di reintrodurre le stopword senza gli scontri tra le due lingue. La rilevazione è già fattibile (sezione 4.4); servirebbero un campo per lingua e l'analisi della query con entrambi gli analyzer.
- **Elisioni:** completare l'elenco degli articoli (*quell'*, *quest'*, *quand'*, *com'*, *pover'*, *sant'*, ...) per recuperare i file mancanti nelle query su `anno`, `uomo` ed `era`, che la valutazione sul corpus grande ha individuato.
- **Indirizzi web nel testo:** un tokenizer che spezzi anche sul punto tra lettere (o un filtro dedicato) recupererebbe casi come `volcano.und`.

- **Verità tematica:** per valutare anche l'ordine dei risultati (precision@5, MAP) si potrebbero usare le categorie di Wikipedia già presenti nel manifest come insieme dei documenti rilevanti per un argomento.
- **Numeri nei nomi:** `nome example` non trova `example1`. Un filtro che separi lettere e numeri *e* conservi il termine originale (opzione `preserveOriginal` del `WordDelimiterGraphFilter`) coprirebbe entrambi i casi.
- **AND come operatore predefinito:** renderebbe più precise le query con più termini, ma rischia di non trovare nulla; andrebbe valutato su un insieme di query con risultati attesi (ad esempio con la precision@5).
- **Caratteri speciali:** il `QueryParser` interpreta ancora la sintassi di Lucene (`campo:valore`, `+`, `-`, `*`), per cui `nome foo:bar` cerca nel campo `foo`. Un parser scritto a mano o l'escape dei caratteri speciali limiterebbero la sintassi a quella richiesta.
- **Test automatici:** i casi della tabella 12 potrebbero diventare test JUnit.

10 Come riprodurre i risultati

```
./index.sh                  # indicizza data/docs nella cartella index/
./search.sh                 # ricerca interattiva (exit per uscire)
./run-queries.sh            # esegue queries.txt e salva results/query-results.md
```

Per rigenerare il corpus grande, misurare i tempi e rifare la valutazione:

```
python3 tools/build_corpus.py          # scarica il corpus in data/corpus (
    ripartibile)
python3 tools/benchmark_indexing.py    # tempi su 200/500/1000/2000/tutti i file
# valutazione (Evaluator si lancia con java e il classpath di Maven):
./mvnw -q dependency:build-classpath -Dmdep.outputFile=cp.txt
java -cp "target/classes:$(cat cp.txt)" it.homework.lucene.Evaluator \
     data/corpus results/evaluation.md
python3 tools/crosscheck_oracle.py     # controllo dell'oracolo con perl e grep
```